Modelos de Negocio en Software Libre y Análisis de Lock-In

PrioritarIA

EIA - Anexo de Modelos de Negocio en Software Libre y Análisis de Lock-In

Índice

1. Punto de partida..... 3
2. Modelo elegido: servicios y soporte sobre un núcleo EUPL-1.2..... 3
3. Por qué encaja con PrioritarIA..... 3
4. Comparación con los modelos que no elegimos..... 4
5. Riesgos de lock-in..... 5
6. Riesgos de conflicto con la comunidad..... 5
7. Límites éticos del modelo de negocio..... 6
Referencias..... 6

1. Punto de partida

PrioritarIA no vende un producto a ciudadanos, sino un sistema de priorización a ayuntamientos. Por eso el modelo de negocio tiene que encajar con un comprador público, con contratos, auditorías y la obligación de explicar cada decisión.

Este apéndice parte de lo que el equipo ya ha decidido en el anexo de componentes y licencias del Sprint 4:

- Licencia del código: EUPL-1.2: cubre todo el código propio: motor de puntuación, filtro de reglas, panel del trabajador social, formularios y pruebas automáticas de varianza y equidad. Es copyleft: quien modifique el código y lo distribuya, o lo ofrezca como servicio, debe hacerlo bajo las mismas condiciones.
- Lo que no es código libre: los datos de los ayuntamientos (nunca se publican), los expedientes y el baremo concreto de cada programa. El baremo lo aprueba y publica el órgano de baremo, pero es una decisión administrativa del ayuntamiento, no un activo del proveedor.

Como la licencia es copyleft, el modelo no puede apoyarse en vender una versión cerrada del software. Si el equipo cambiara a una licencia permisiva (MIT o Apache-2.0), habría que revisar la comparación de la sección 4 y el riesgo de apropiación de la sección 6.

2. Modelo elegido: servicios y soporte sobre un núcleo EUPL-1.2

Elegimos un modelo de servicios profesionales y soporte: el código es libre y gratuito, y la empresa cobra por lo que un ayuntamiento no puede ni debe hacer solo. Lo que se cobra es lo siguiente:

- Despliegue e integración: con la sede electrónica y los registros municipales. Requiere conocer cada ayuntamiento y la interoperabilidad con otras administraciones.
- Mantenimiento y soporte con acuerdo de nivel de servicio: el proveedor es quien debe corregir los fallos técnicos (Sprint 3, Apéndice F).
- Formación a trabajadores sociales: para evitar el sesgo de automatización hace falta que sepan leer el vector SHAP.
- Revisión periódica de pesos y auditoría de sesgos: los pesos se reajustan offline, se validan y se congelan antes de desplegarse.
- Apoyo documental para el AI Act y la DPIA: es un sistema de alto riesgo con obligaciones de documentación y trazabilidad.

No se cobra la licencia de uso, ni el número de expedientes, ni el acceso al código. Tampoco se limita el uso del software a quien contrate el soporte: un ayuntamiento puede desplegarlo con sus propios técnicos o con otra empresa.

El contrato tipo sería una cuota anual de soporte y mantenimiento, más proyectos puntuales de despliegue, formación y auditoría. Los ingresos dependen de que el ayuntamiento esté satisfecho, no de tenerlo atado.

3. Por qué encaja con PrioritarIA

- El comprador es público y ya está empujado hacia software reutilizable: el artículo 157 de la Ley 40/2015 obliga a las administraciones a consultar el directorio

general de aplicaciones antes de adquirir o desarrollar una y a reutilizar las soluciones disponibles. Prevé además que las aplicaciones puedan declararse de fuentes abiertas cuando favorezca la transparencia del funcionamiento administrativo [1]. Un núcleo con licencia libre se puede ceder entre ayuntamientos sin renegociar licencias.

- La transparencia es el argumento central del proyecto: BOSCO se criticó por ser una caja negra no auditable y SyRI se declaró ilegal por falta de transparencia (Apéndice A). Con código abierto, la auditoría no depende de que el proveedor colabore.
- Encaja con las obligaciones de alto riesgo: como sistema de alto riesgo, PrioritarIA ya debe documentar, registrar y permitir la supervisión humana. Vender servicios de documentación y auditoría convierte ese coste regulatorio en un servicio concreto.
- Reparte bien la responsabilidad: en el Sprint 3 fijamos que responde el ayuntamiento y que el proveedor corrige los fallos técnicos. Un contrato de soporte con acuerdo de nivel de servicio recoge justo esa obligación, y el código abierto permite comprobar si el fallo es del software o de su uso.
- Por qué EUPL-1.2: cumple los criterios del equipo: es copyleft (nadie puede cerrar una versión modificada), contempla el uso como servicio, que es como se despliega PrioritarIA, y es una licencia pensada para el sector público europeo, con validez en el derecho español y de la UE. Evita el escenario en que un tercero mejora el sistema en privado y lo ofrece a otros ayuntamientos sin compartir los cambios. Sus dependencias principales (XGBoost, SHAP) tienen licencias abiertas que se pueden incorporar a una obra EUPL sin problema [2].

4. Comparación con los modelos que no elegimos

Comparamos el modelo elegido con tres alternativas habituales en software libre. Las tres funcionan en otros sectores, pero chocan con algo que ya prometimos en los sprints anteriores.

- Servicios y soporte (elegido): se cobra por despliegue, soporte, formación y auditoría. Encaja con el comprador público y con la transparencia exigida. Su riesgo principal son los ingresos irregulares y la dependencia de pocos clientes grandes.
- Open core: núcleo libre y módulos de pago cerrados. Lo descartamos porque lo que se vendería como extra (explicabilidad, auditoría, pruebas de equidad) es justo lo que un sistema de alto riesgo debe tener abierto. El riesgo sería una versión gratuita incompleta y un cliente obligado a pagar por cumplir la ley.
- Doble licencia: se vende una licencia comercial a quien no quiera copyleft. Lo descartamos porque vender la excepción permitiría cerrar versiones de un sistema que prioriza personas vulnerables, contra el criterio de que no se pueda cerrar. Además exige concentrar la titularidad del código y desanima a contribuir.
- SaaS gestionado por el proveedor: la empresa aloja y opera el sistema para varios ayuntamientos. Lo descartamos porque concentraría datos de salud, menores y renta de muchos ayuntamientos en un único proveedor: un fallo o una brecha afectaría a todos a la vez y el cliente quedaría atado a la infraestructura.

El servicio gestionado no queda prohibido: un ayuntamiento puede contratar el alojamiento a cualquier proveedor, incluida nuestra empresa. Lo que descartamos es que sea la base del negocio.

5. Riesgos de lock-in

El código libre no elimina por sí solo la dependencia del proveedor: se puede estar atado por el conocimiento, los datos o el contrato aunque el código sea público. Estos son los riesgos que vemos y cómo los reducimos.

- Dependencia del conocimiento: aparece si solo el proveedor sabe mantener el sistema. Medida: documentación técnica pública en el repositorio y formación a los técnicos del ayuntamiento.
- Dependencia de los datos: aparece si expedientes, perfiles y registros de auditoría están en formatos cerrados. Medida: exportación en formatos abiertos (CSV, JSON) y API documentada.
- Dependencia del modelo: aparece si solo el proveedor puede reajustar los pesos. Medida: pesos y baremo son del ayuntamiento, se publican y el procedimiento de reajuste offline está en el repositorio.
- Dependencia del contrato: aparece con renovaciones automáticas y costes de salida. Medida: contratos de duración limitada con cláusula de salida (entrega de datos, documentación y apoyo en la transición).
- Dependencia de la infraestructura: aparece si se imponen servicios propietarios. Medida: despliegue en la infraestructura del ayuntamiento, sin servicios de nube propietarios imprescindibles.
- Dependencia de terceros: aparece si una librería cambia de licencia o se abandona. Medida: versiones fijadas y todas las dependencias inventariadas en el BOM (Apéndice J).

La prueba de que el lock-in está bajo control es sencilla: otra empresa, o el propio ayuntamiento, debe poder tomar el repositorio, la documentación y una exportación de datos y mantener el sistema sin nuestra ayuda.

6. Riesgos de conflicto con la comunidad

Un proyecto libre vive de que otras personas y organizaciones confíen en él. Estos son los puntos donde podría haber conflicto.

- Que una gran empresa lo ofrezca como servicio sin devolver nada: la EUPL cubre el uso como servicio, así que las versiones modificadas deben seguir siendo libres.
- Que cambiemos de licencia más adelante: nos comprometemos a no pasar a una licencia no libre. El caso de MongoDB muestra el coste: su cambio a la SSPL recibió críticas por contradecir los principios del software libre [3].
- Que otras empresas compitan vendiendo soporte: lo aceptamos como parte del modelo: nadie tiene exclusividad y nuestro valor es conocer el sistema mejor que nadie.
- Que alguien haga un fork que quite salvaguardas (segunda firma, bloqueo sin SHAP, auditoría): la licencia no puede impedirlo. Las pruebas de equidad viajan con el

código, así que cualquier fork las incluye. Una guía de uso de marca (tarea de marcas y naming) podría impedir que un fork sin estas salvaguardas use el nombre PrioritarIA.

- Que contribuciones externas introduzcan sesgos o fallos de seguridad: toda contribución al motor de puntuación exige revisión de dos personas y pasar las pruebas de varianza y equidad.

Las contribuciones externas se aceptan bajo la misma licencia que el proyecto, sin ceder derechos de relicenciación a la empresa.

7. Límites éticos del modelo de negocio

Un sistema que prioriza a familias vulnerables no puede tener un modelo de negocio que empuje en contra de ellas. Fijamos cuatro límites, coherentes con las líneas rojas del Sprint 3:

1. Los ingresos no dependen de lo que decida el modelo: no se cobra por expediente priorizado, ni por ayudas denegadas, ni por «ahorro» conseguido. Solo se cobra por servicios, no por resultados.
2. Quien mantiene no es el único que audita: las auditorías de sesgo y equidad se pueden encargar a un tercero independiente o a una ONG, y los informes se entregan al órgano de baremo del ayuntamiento.
3. Los datos no son un activo: el proveedor no usa los datos de un ayuntamiento para entrenar productos para otros ni los conserva tras el contrato, en línea con la limitación de la finalidad del RGPD.
4. Los pesos y el baremo solo cambian por decisión del órgano de baremo: ningún contrato de soporte permite al proveedor retocar la puntuación por su cuenta, y el sistema sigue sin usar IA generativa.

Referencias

1. Ley 40/2015, artículo 157, reutilización de sistemas y aplicaciones de propiedad de la Administración.
2. Comisión Europea, EUPL compatible open source licences.
3. The Register, cambio de licencia de MongoDB (SSPL), 2018.
4. PrioritarIA, Anexo - Componentes del Sistema y sus Licencias v1.0 (Sprint 4).
5. PrioritarIA, Sprint 3: Anexos de Definición del proyecto v3.0, Líneas rojas, Emergencias y Rendición de cuentas y reparación.